Fiche orale — Agent Purchase DressMe

ESPRIT · Qwen2.5-1.5B-Instruct + LoRA · 5 outils · MCP
Phrases courtes, à la première personne. À lire à voix haute.

> **Dire en premier.** Le score d'achat n'est pas le LLM. Le LLM appelle des outils et rédige. La fidélité est à 0 %. Le parseur aide le format. MCP n'est pas encore dans la boucle de l'agent. La loss Colab n'est pas dans le notebook Git.

Partie 1 — termes A à E

A. Agent et orchestration

1. Définition

Un agent, c'est un programme qui agit pour un but précis.

L'orchestration, c'est le chef d'orchestre : il dit qui joue, et quand.

2. Comment j'ai pensé

Le problème : la vision, le score et le texte ne doivent pas se mélanger.

J'ai choisi un chef plus un agent d'achat.

J'ai écarté un seul gros modèle pour tout : trop opaque.

Le score reste une formule. Le petit LLM appelle les outils, puis rédige.

3. Comment dans le code

`evaluate_purchase` lance la vision, puis la garde-robe, puis l'achat.

`PurchaseAgent.run` calcule le score avec quatre poids fixes.

Ensuite `run_agent` peut remplacer seulement le texte d'explication.

4. Où je montre

`backend/app/agents/orchestrator.py` 37–50 et 220–254.

`backend/app/agents/purchase_agent.py` 22 (poids 40 / 25 / 20 / 15) et 127–178.

5. Ce que je dis

Ici, je ne laisse pas le LLM décider le score. Le score est une moyenne pondérée, toujours la même. Le LoRA n'a le droit que d'appeler des outils. Puis il rédige deux à quatre phrases en français. Si le LoRA n'a pas tourné, un autre agent peut expliquer. Le chef d'orchestre, c'est `evaluate_purchase`.

6. Si elle me questionne

Le LLM choisit-il d'acheter ? Non. Le score est calculé avant, sans lui.

Piège : alors pourquoi un LLM ? Pour le texte et les appels d'outils, pas le verdict. Limite : si le LoRA échoue, le score reste, le texte redevient `_explain`.

B. LLM choisi pour chaque agent

1. Définition

Un LLM, c'est un modèle qui prédit le mot suivant, comme un clavier intelligent.

FashionCLIP et SegFormer regardent des pixels. Ce ne sont pas des LLM.

2. Comment j'ai pensé

Le problème : j'avais un GPU T4 gratuit, pas une grosse machine.

J'ai choisi Qwen2.5-1.5B-Instruct : petit, français, licence Apache.

J'ai écarté GPT-4 pour l'achat : payant, et je ne peux pas le fine-tuner ici.

Pour la vision, j'ai écarté un LLM : une image n'est pas du texte.

3. Comment dans le code

Purchase charge Qwen plus l'adapter LoRA.

La vision charge FashionCLIP, puis SegFormer pour découper le vêtement.

LLMAgent parle à une API : Groq, Gemini, Ollama, OpenAI, Mistral, ou rien.

4. Où je montre

`purchase_llm.py` 27 · `finetune_purchase_llm.py` 7 (T4) · `vision_model.py` 40, 100–107 · `garment_segmentation.py` 23, 74–78 · `llm_agent.py` 40–47 · `config.py` 40 (none).

5. Ce que je dis

Pour l'achat, j'ai pris Qwen un virgule cinq milliards de paramètres. Il tient sur un T4 Colab gratuit. La vision n'est pas un LLM : FashionCLIP encode, SegFormer découpe. Les textes du profil passent par LLMAgent, une API au choix. Par défaut cette API est éteinte. Donc Purchase peut tourner tout seul, en local, avec LoRA.

6. Si elle me questionne

Pourquoi pas Llama 70B ? Trop gros pour T4. Je n'aurais pas pu fine-tuner.

Piège : 1.5B n'est-il pas trop faible ? Oui, il hallucine facilement. C'est pour ça que le sandbox bloque les outils, et que le score n'est pas au LLM.

C. Prompt engineering

1. Définition

Le prompt, c'est la consigne écrite qu'on donne au modèle.

Bien le rédiger, c'est comme un brief clair à un stagiaire.

2. Comment j'ai pensé

Le problème : le modèle invente des prix s'il n'a pas de cadre.

J'ai mis un rôle, une interdiction, une liste d'outils, un format, un ordre.

J'ai écarté le few-shot à l'inférence : ça allonge le prompt.

Les exemples sont dans l'entraînement, pas recopiés à chaque achat.

3. Comment dans le code

`SYSTEM_PROMPT` fixe le rôle et l'interdiction d'inventer.

`_TOOL_LIST` injecte les cinq noms depuis `TOOL_SPECS`.

Le format impose `<tool_call>` puis `<final>`.

L'entrée user est un JSON. La boucle outil puis résultat, c'est du ReAct.

4. Où je montre

`backend/app/ml/purchase_llm.py` 30–46, 94–97, 100–116.

5. Ce que je dis

Je commence par : tu es l'agent d'achat DressMe. Tu n'as pas le droit d'inventer un prix, un score, une marque. Je colle la liste des cinq outils, rien d'autre. Tu dois répondre dans des balises, une action à la fois. L'ordre : garde-robe, catalogue, prix, marques, coût par port, puis final. Je n'ajoute pas d'exemples au runtime : ils sont déjà dans le fine-tuning.

6. Si elle me questionne

C'est quoi ReAct ? Agir, voir le résultat, puis agir encore, jusqu'à la réponse.

Piège : le prompt suffit-il sans LoRA ? Non. Sans adapter, le petit Qwen rate souvent les balises.

D. Tool schema

1. Définition

Un tool schema, c'est la fiche d'un outil : nom, rôle, parfois des arguments.

Comme la notice d'un appareil : ce qu'il fait, et quels boutons.

2. Comment j'ai pensé

Le problème : le LLM ne doit voir que cinq actions, pas Python.

J'ai choisi un schéma minimal : nom plus une phrase.

J'ai écarté un gros JSON Schema : quatre outils n'ont aucun argument.

MCP ajoute le schéma là où il y a un argument : `limit`.

3. Comment dans le code

`TOOL_SPECS` a cinq dictionnaires : `name` et `description`.

`search_tunisian_catalog` accepte `limit`, défaut vingt.

FastMCP fabrique le schéma JSON de cet argument.

4. Où je montre

`purchase_tools.py` 17–38 et 103 · `purchase_server.py` 45–67.

Le JSON MCP exact de `inputSchema` : à vérifier en lançant la démo.

5. Ce que je dis

Dans le code métier, le schéma est volontairement petit. Nom, plus une phrase, c'est tout. Quatre outils n'ont pas de paramètre. Le cinquième a `limit`, un entier, vingt par défaut. MCP reprend les vrais noms et les vraies descriptions. Il expose `limit` parce que Python l'a typé.

6. Si elle me questionne

Pourquoi pas les types de tous les champs ? Parce que les autres fonctions n'ont pas d'arguments.

Piège : un schéma sans types, c'est faible ? Oui. La vraie porte, c'est la liste blanche, pas le schéma.

E. Sandbox / liste blanche

1. Définition

Une liste blanche, c'est : seuls ces noms-là ont le droit de passer.

Le sandbox, c'est le bac à sable : pas de Python, pas d'internet.

2. Comment j'ai pensé

Le problème : un LLM peut écrire `os_system` ou `eval`.

Le prompt et le LoRA restent probabilistes : ça peut encore arriver.

Le sandbox est déterministe : nom inconnu, erreur, point final.

3. Comment dans le code

`call` cherche le nom dans un dictionnaire des cinq outils.

Si le nom manque, il lève `UnknownPurchaseTool`.

Les vecteurs restent dans le sandbox. Le modèle ne voit que le JSON.

4. Où je montre

`purchase_tools.py` 1–5, 41–42, 53–55, 78–82.

5. Ce que je dis

Le prompt, c'est un conseil. Le modèle peut désobéir. Le fine-tuning, c'est une habitude. Il peut encore se tromper. Le sandbox, c'est une porte fermée à clé. Cinq noms passent. Un sixième nom lève une erreur. Donc la sécurité ne repose pas sur la politesse du modèle.

6. Si elle me questionne

Différence prompt / LoRA / sandbox ? Conseil, habitude, puis interdiction réelle.

Piège : le sandbox valide-t-il les types ? Non. Il valide le nom. C'est une limite.

Partie 2 — termes F à J

F. RAG

1. Définition

RAG, c'est aller chercher une info, puis la coller dans le texte du modèle.

Comme ouvrir un classeur avant de répondre, au lieu d'inventer de mémoire.

2. Comment j'ai pensé

Je veux des produits tunisiens qui se ressemblent, pas des mots.

J'ai choisi un kNN visuel. J'ai écarté un RAG texte dans Purchase.

Le RAG texte appartient à l'agent de recommandation.

3. Comment dans le code

Les produits sont dans `catalog_embeddings.npz`.

`search` fait un produit scalaire, puis prend les meilleurs scores.

Le vecteur n'entre jamais dans le prompt. Les outils renvoient du JSON.

4. Où je montre

`catalog_index.py` 1–7, 23, 80–88, 143–155 · `vision_model.py` 138 (512-d) · `purchase_llm.py` 10 · `context_retrieval.py` 1–3, 53.

5. Ce que je dis

Chez moi, ce n'est pas un RAG classique avec des phrases. C'est un kNN visuel : je cherche les habits qui se ressemblent. FashionCLIP donne un vecteur de cinq cent douze nombres. Le LLM ne voit que les JSON des outils, jamais le vecteur. Le vrai RAG texte, c'est l'autre agent, Recommendation.

6. Si elle me questionne

C'est bien du RAG ? C'est de la récupération, oui, mais visuelle, pas textuelle.

Piège : pourquoi dire RAG ? On injecte le résultat des outils. Ce n'est pas un index de documents. Je le dis franchement.

G. Fine-tuning

1. Définition

Fine-tuner, c'est ré-entraîner un modèle déjà savant sur mon petit métier.

Comme un cuisinier diplômé qui apprend ensuite ma carte.

2. Comment j'ai pensé

Qwen ne connaît pas mes balises ni mes cinq outils.

Full fine-tuning : trop lourd. Prompt tuning : trop faible pour le ReAct.

LoRA : petites matrices, Qwen gelé, ça tient sur T4.

3. Comment dans le code

`save_pretrained` sauve l'adapter, pas tout Qwen.

`PeftModel.from_pretrained` pose l'adapter par-dessus la base.

Je ne fusionne pas. Je peux éteindre l'adapter pour comparer.

4. Où je montre

`finetune_purchase_llm.py` 2–4, 42–46, 61 · `purchase_llm.py` 170–172 · `eval_purchase.py` 10, 296 (`disable_adapter`).

5. Ce que je dis

Je n'ai pas réécrit Qwen. J'ai collé un petit adaptateur LoRA. L'adapter est un fichier à part. Je charge Qwen, puis je pose l'adapter par-dessus. Je ne merge pas. Je peux comparer avec et sans. Le score d'achat ne passe pas par cet entraînement.

6. Si elle me questionne

Pourquoi pas tout ré-entraîner ? Pas de VRAM, et je perdrais le français général.

Piège : LoRA a-t-il appris les prix ? Non. Il a appris le format d'appel. Les chiffres viennent des outils.

H. Dataset SFT

1. Définition

SFT, c'est l'apprentissage par imitation : je montre le dialogue idéal.

Comme un script de théâtre : le modèle répète les répliques justes.

2. Comment j'ai pensé

Je n'avais pas vingt-cinq vrais achats annotés.

J'ai écrit des trajectoires synthétiques. Je assume : petit, répétitif, fautes d'accord.

3. Comment dans le code

`teacher_trajectory` fabrique system, user, outils, puis `<final>`.

Vingt-quatre cas : six catégories, quatre scénarios. Plus un hors vêtement. Total 25.

Accessoire n'est pas dans l'entraînement.

4. Où je montre

`purchase_llm_sft.jsonl` : 25 lignes. Ligne 2 = haut noir 59. Ligne 20 = « ces chaussures crée ».

`build_purchase_llm_dataset.py` 17–38.

5. Ce que je dis

Chaque ligne est un dialogue complet. Vingt-quatre fois le même schéma, six habits, quatre histoires. Noir 59, beige 89, blanc 75, rouge 95. Plus un cas « ce n'est pas un vêtement ». C'est synthétique. Il y a même « ces chaussures crée ». Je le montre, je ne le cache pas.

6. Si elle me questionne

Pourquoi 25 ? Assez pour le format, trop peu pour un style riche.

Piège : il a juste mémorisé ? Risque oui. D'où dix cas d'éval hors de ces prix. Limite : les traces se ressemblent encore.

I. Hyperparamètres

Réglage	Où	C'est quoi	Pourquoi
r = 16	finetune 43	Taille des petites matrices LoRA	Assez pour le format, pas trop de poids
alpha = 32	43	Force LoRA (alpha/r = 2)	Classique, un peu fort
dropout = 0.05	43	J'éteins 5 % des liens LoRA	Moins coller aux 25 exemples

q/k/v/o_proj	44	J'adapte l'attention	Là où le modèle choisit le mot
lr 2e-4	54	Vitesse des pas	LoRA supporte plus vite que du full
batch 2 × accum 4 = 8	51–52	Même effet qu'un batch 8	Moins de mémoire
époques = 10	53 ; notebook 143	Dix lectures des 25 lignes	Voir terme J
logging_steps = 10	55	J'affiche la loss tous les 10 pas	Voir si ça descend
fp16 = False	58	Calcul en float32	Plus stable sur 1.5B
max_new_tokens = 256	purchase_llm 183	Au plus 256 mots nouveaux	160 coupait les balises
do_sample = False	183	Greedy : mot le plus probable	Reproductible
temperature = 0.2	183	Devrait adoucir le hasard	**Sans effet** si greedy

Piège : j'ai mis temperature 0.2, mais `do_sample=False` l'annule. Je le dis avant le warning.

J. Époques

Une époque, c'est un tour complet des 25 dialogues.

Le script et le notebook du projet disent **10**.

L'entraînement réel : Colab, T4. Preuve = le fichier Colab téléchargé.

> **Écart.** Le `.ipynb` du repo a des sorties vides. La loss 6,10 → 0,63 n'est **pas** dans Git. Je ne l'invente pas. Je la montre seulement si j'ai l'export Colab.

Partie 3 — termes K à O

K. Chargement du modèle

1. Définition

Charger le modèle, c'est l'ouvrir en mémoire pour qu'il puisse écrire.

`lru_cache`, c'est : je le charge une fois, puis je le réutilise.

2. Comment j'ai pensé

Le problème : Qwen pèse des giga. Je ne dois pas le recharger à chaque phrase.

J'ai choisi float32 CPU dans le code de prod. Pas de `cuda` dans ce pipeline.

J'ai écarté de fusionner LoRA : je veux pouvoir l'éteindre.

3. Comment dans le code

`_local_pipeline` est décoré `@lru_cache`.

Il charge le tokenizer, Qwen en float32, puis l'adapter, puis `eval()`.

`eval` veut dire : plus d'apprentissage, seulement de l'écriture.

4. Où je montre

`backend/app/ml/purchase_llm.py` 161–173, 176–184.

À l'écran : `torch.float32`, `PeftModel.from_pretrained`, pas de `.to("cuda")`.

5. Ce que je dis

Je charge Qwen une seule fois, grâce au cache. Tokenizer, puis le gros modèle en float32. Puis je pose l'adapter LoRA. Puis j'appelle `eval` : on n'apprend plus. Sur ma machine, ça tourne CPU. C'est lent, mais le code de prod ne force pas le GPU.

6. Si elle me questionne

Pourquoi float32 ? Plus simple, plus stable, et le script d'entraînement est aussi en fp16 False.

Piège : ça utilise le GPU ? Le chargeur local ne déplace pas le modèle sur CUDA. Limite : inférence CPU, lente.

L. Boucle agent et parseur

1. Définition

La boucle, c'est : le modèle parle, j'exécute, je lui rends le résultat.

Le parseur lit les balises et en tire un nom d'outil, ou un texte final.

2. Comment j'ai pensé

Sans parseur, le texte du modèle ne sert à rien.

J'ai limité à six tours, pour éviter une boucle infinie.

J'ai assoupli les balises fermantes : rustine, pas un beau design.

3. Comment dans le code

`run_agent` génère, parse, appelle `sandbox.call`, renvoie `<tool_result>`.

Au plus six fois. Puis `<final>`, ou rien.

Les regex acceptent une balise sans fermeture.

4. Où je montre

`purchase_llm.py` 28, 56–57, 60–82, 85–117.

À l'écran : `(?:</tool_call>)?` et `MAX_TOOL_STEPS = 6`.

5. Ce que je dis

Le modèle écrit une balise. Je la lis. Si c'est un outil, le sandbox l'exécute. Je lui rends le JSON. Il recommence, au plus six fois. Puis il doit écrire un final. J'accepte même une balise non fermée. Donc le 100 % de format est un peu aidé par le parseur. Je le dis moi-même.

6. Si elle me questionne

Pourquoi six ? Cinq outils plus le final. Un cran de marge, pas une boucle libre.

Piège : le 100 % est-il honnête ? Le format est bon, mais le parseur est tolérant. C'est une limite.

M. Évaluation

1. Définition

Évaluer, c'est noter le modèle sur des cas qu'il n'a pas appris par cœur.

Trois notes : le format, le choix d'outils, la fidélité des chiffres.

2. Comment j'ai pensé

Les 25 trajectoires ne suffisent pas comme preuve.

J'ai fait 10 cas différents : autres prix, autres couleurs, accessoire.

Je compare LoRA et base en éteignant l'adapter, sans second modèle.

3. Comment dans le code

Format : balises valides, et un `<final>` à la fin.

Outils : noms autorisés, et `inspect_wardrobe` en premier.

Fidélité : les TND et la médiane doivent matcher les outils.

4. Où je montre

`eval_purchase.py` 46–57 (cas), 148–174, 173–174, 219–240.

Résultat LoRA lu dans le log : format 100 %, outils 100 %, fidélité 0 %.

Les 10 traces sont la même suite de cinq outils.

Tableau base : pas dans le dépôt. Je ne l'ai pas fini.

5. Ce que je dis

J'ai testé dix achats hors du fichier d'entraînement. Avec LoRA, le format et les outils sont bons. La fidélité est à zéro : les chiffres de l'explication ne collent pas assez. Les dix traces sont identiques. Je n'ai pas encore le tableau sans LoRA dans le repo. Je le dis avant qu'elle le demande.

6. Si elle me questionne

Pourquoi 0 % fidélité ? Le modèle appelle bien les outils, mais il recopie mal les médianes.

Piège : 100 % outils, c'est trop beau ? Les dix cas ont la même trace. Ça ressemble à une habitude, pas à dix raisonnements.

N. MCP

1. Définition

MCP, c'est un tuyau standard pour exposer des outils à un modèle.

Comme une prise USB : le même format, plusieurs appareils.

2. Comment j'ai pensé

La prof a demandé comment le LLM parle aux tools, exemple MCP.

J'ai wrappé le sandbox existant. Je n'ai pas réécrit la boucle ReAct.

Version `mcp[cli]<2` : l'API FastMCP v1. La v2 l'a cassée.

3. Comment dans le code

Qwen écrit `<tool_call>`. `parse_action` lit le nom.

Le client MCP envoie. Le serveur FastMCP appelle `sandbox.call`.

La boucle réelle de l'agent passe encore par le sandbox direct.

MCP se voit avec `demo_mcp`, pas dans `run_agent`.

4. Où je montre

`purchase_server.py` 17, 45–71, 1–4 · `purchase_client.py` · `demo_mcp.py` 32–79 · `docs/mcp.md` · `requirements.txt` 20.

Démo : a) liste et schéma · b) cinq appels · c) parse_action puis outil inconnu.

`isError true` : à vérifier à l'écran. Le code imprime une exception ou un dump d'erreur. Je n'affirme pas le nom du champ sans l'avoir relancé.

5. Ce que je dis

MCP, c'est le standard pour brancher des outils. Qwen écrit une balise. Le parseur lit le nom. Le client envoie au serveur. Le serveur appelle le même sandbox. L'outil inconnu est refusé. Attention : la boucle de l'agent, aujourd'hui, n'passe pas encore par MCP. Je le démontre avec le script de démo. J'ai pinné mcp inférieur à 2.

6. Si elle me questionne

L'app utilise MCP en prod ? Pas encore. `run_agent` appelle encore `sandbox.call`.

Piège : alors c'est du vent ? Non. C'est la même liste blanche, derrière un protocole. La limite, c'est que ce n'est pas branché dans la boucle.

O. Security

1. Définition

Ici, sécurité veut dire : le modèle ne peut pas tout faire.

Pas de code libre, pas de web, pas de clé affichée.

2. Comment j'ai pensé

E2B sert si le modèle exécute du code dans une machine jetable.

Chez moi, il n'exécute pas de code. Il choisit un nom parmi cinq.

Donc E2B ne m'apporte rien. La liste blanche suffit.

3. Comment dans le code

Liste blanche : `UnknownPurchaseTool`.

Au plus six appels. Interrupteur `purchase_llm_enabled`.

La clé API est masquée : `repr=False`.

Timeouts : 8 secondes pour LLMAgent. 30 secondes pour l'API Purchase de secours. 20 secondes pour le chat.

Il manque la validation des types d'arguments.

4. Où je montre

Quoi	Où	Valeur
Liste blanche	purchase_tools.py 78–82	nom hors liste → erreur
MAX_TOOL_STEPS	purchase_llm.py 28, 100	6
purchase_llm_enabled	config.py 46 ; purchase_llm.py 88–89	true par défaut
clé API masquée	config.py 39	repr=False
timeout LLMAgent	config.py 51 ; llm_agent.py 273–274	8 s
timeout chat	config.py 50	20 s
timeout API Purchase	purchase_llm.py 198	30 s (secours HTTP)
Qwen local	purchase_llm.py 176–184	pas de timeout HTTP

5. Ce que je dis

Le modèle n'a pas Python. Cinq noms, et c'est tout. Six tours maximum. Je peux couper le LLM Purchase avec un booléen. La clé API n'apparaît pas dans les logs. Huit secondes pour LLMAgent. Trente secondes seulement si Purchase passe par une API. En local, pas d'HTTP. E2B, je n'en ai pas besoin : je n'exécute pas de code libre. Ce qui manque : vérifier les types des arguments.

6. Si elle me questionne

Pourquoi pas E2B ? E2B isole du code. Ici il n'y a pas de code à isoler.

Piège : 8 s ou 30 s ? Les deux. 8 s = LLMAgent. 30 s = secours Purchase HTTP. Le LoRA local n'a pas ce timeout.

Pitch de 3 minutes

DressMe aide à décider si un habit vaut le budget. Moi, j'ai l'agent d'achat.

Un chef d'orchestre lance la vision, puis la garde-robe, puis mon agent.

Le score est une formule. Quarante, vingt-cinq, vingt, quinze. Pas le LLM.

La vision, ce n'est pas un LLM. FashionCLIP encode. SegFormer découpe.

Pour le texte d'achat, j'ai pris Qwen un virgule cinq milliards. GPU T4 gratuit.

Je lui ai écrit un brief : tu n'inventes rien, tu appelles cinq outils, tu respectes les balises.

Pas d'exemples au runtime. Les exemples sont dans le fine-tuning.

Cinq fiches d'outils, volontairement petites. Un seul argument : limit.

Le prompt conseille. Le LoRA habitue. Le sandbox interdit. C'est déterministe.

Ce n'est pas un RAG texte. C'est un kNN visuel sur le catalogue tunisien.

Le vecteur n'entre jamais dans le prompt. Seulement le JSON des outils.

J'ai fine-tuné avec LoRA. Adapter à part. Je peux comparer avec et sans.

Vingt-cinq dialogues synthétiques. Six catégories, quatre scénarios, plus hors vêtement.

C'est répétitif. Il y a « ces chaussures crée ». Je le assume.

Réglages : r seize, alpha trente-deux, batch deux fois quatre, dix époques.

Greedy à l'inférence. La température est dans le code, mais sans effet.

La loss Colab n'est pas dans le notebook Git. Je le dis.

Je charge Qwen une fois, float32, adapter par-dessus, mode eval.

Boucle : balise, outil, résultat, au plus six fois, puis final.

Le parseur accepte une balise non fermée. Le 100 % de format est un peu aidé.

Dix cas hors SFT : format 100, outils 100, fidélité 0. Traces identiques.

Pas encore de tableau base dans le repo.

MCP : même sandbox, autre tuyau. Démo en trois étapes. Pas encore dans la boucle.

Sécurité : liste blanche, six tours, clé masquée, timeouts 8 et 30. Pas E2B.

Voilà ce que j'ai fait, et voilà ce qui reste faible.

Dialogue type

1. **Prof :** Comment as-tu fait ?
   Moi : Score en formule. LoRA pour les outils et le texte. Sandbox fermé.
2. **Prof :** Ok, montre-moi.
   Moi : J'ouvre `purchase_agent.py` ligne 127, le score, puis 171, `run_agent`.
3. **Prof :** C'est quoi LoRA ?
   Moi : De petites matrices collées sur Qwen. Je n'ai pas réécrit le gros modèle.
4. **Prof :** Montre les outils.
   Moi : J'ouvre `purchase_tools.py` ligne 17, les cinq noms, puis 78, la liste blanche.
5. **Prof :** C'est quoi MCP ?
   Moi : Un tuyau standard vers les mêmes outils. Démo dans `demo_mcp.py`. Pas encore dans la boucle.
6. **Prof :** Tes résultats ?
   Moi : J'ouvre `eval_purchase.py` ligne 46. Dix cas. Format et outils 100. Fidélité 0. Je l'assume.

Les 8 limites à dire AVANT qu'elle les trouve

#	Limite	Je dis	Ensuite
1	Fidélité 0 %	Les outils marchent. Les chiffres de l'explication, pas encore.	Forcer la médiane dans le `<final>`, ou plus d'exemples chiffrés.
2	Parseur tolérant	Le 100 % de format est un peu aidé par le parseur.	Revenir à des balises strictes après un meilleur LoRA.
3	25 exemples synthétiques	C'est un script, pas 25 vrais achats.	Annoter de vrais cas, et casser les copies-collées.
4	loss sur toute la séquence	`labels = input_ids` : le modèle apprend aussi le padding.	Masquer tout sauf la réponse assistant.
5	Température ignorée	0.2 est dans le code. Greedy l'annule.	Enlever temperature, ou activer do_sample pour de vrai.
6	Pas de tableau base	Je n'ai pas le comparatif sans LoRA dans le repo.	Relancer `--mode base` et coller le tableau.
7	Schémas sans types	TOOL_SPECS = nom + phrase. MCP n'ajoute que limit.	Déclarer les types de tous les arguments.
8	MCP hors boucle	La démo marche. `run_agent` appelle encore le sandbox direct.	Remplacer `sandbox.call` par le client MCP, sans changer le ReAct.

Ordre d’ouverture des fichiers

1. `backend/app/agents/orchestrator.py` — 37–50, 220–254
2. `backend/app/agents/purchase_agent.py` — 22, 127–178
3. `backend/app/ml/purchase_llm.py` — 27, 30–46, 56–57, 85–117, 161–184
4. `backend/app/services/purchase_tools.py` — 17–38, 78–82
5. `backend/app/data/purchase_llm_sft.jsonl` — lignes 2 et 20
6. `backend/scripts/finetune_purchase_llm.py` — 42–61
7. `backend/scripts/eval_purchase.py` — 46–57, 148–174, 219–240
8. `backend/app/mcp/purchase_server.py` — 17, 45–71
9. `backend/scripts/demo_mcp.py` — 32–79
10. `backend/app/services/catalog_index.py` — 23, 80–88
11. `backend/app/core/config.py` — 39–51
12. `ml/notebooks/14_purchase_llm_lora.py` — 143 (10 époques ; pas de loss dans Git)

Mini-glossaire (25 termes)

LLM	Modèle qui prédit le mot suivant.
Token	Un morceau de mot que le modèle lit ou écrit.
Agent	Programme qui agit pour un but, pas seulement qui classe.
ReAct	Agir, voir le résultat, puis agir encore.
LoRA	Petites matrices ajoutées, le gros modèle reste gelé.
Adapter	Le fichier de ces petites matrices, à part.
Rang (r)	Taille de ces petites matrices. Chez moi, 16.
Alpha	Force de LoRA. Chez moi 32, donc alpha/r = 2.
Dropout	J'éteins au hasard 5 % des liens pendant l'apprentissage.
Epoch	Un passage complet sur les 25 dialogues.
Batch	Combien d'exemples je vois avant de corriger les poids.
Learning rate	Taille du pas de correction. Chez moi 2e-4.
Loss	L'erreur : plus elle baisse, plus le modèle imite le script.
SFT	Je montre le dialogue idéal, le modèle imite.
Greedy	Je prends toujours le mot le plus probable.
Température	Dose de hasard. Ici sans effet, car greedy.
Embedding	Une liste de nombres qui représente une image ou un mot.
kNN	Je prends les plus proches voisins dans cet espace.
RAG	Chercher une info, puis la donner au modèle.
Sandbox	Bac à sable : le modèle n'a ni Python ni web.
Liste blanche	Seuls ces cinq noms passent.
MCP	Tuyuau standard pour exposer des outils.
Tool schema	La fiche d'un outil : nom, phrase, parfois un argument.
Parseur	Il lit les balises et en tire un nom ou un texte.
Hallucination	Le modèle invente un prix ou une marque.

À vérifier et écarts

- Schéma JSON MCP de `limit` : à vérifier en lançant `demo_mcp`.
- Champ `isError` : à vérifier à l'écran. Le code imprime exception ou dump.
- Loss 6,10 → 0,63 : **absente** du notebook Git (sorties vides). Seulement Colab perso.
- Tableau `--mode base` : **absent** du dépôt.
- Timeouts : 8 s = LLMAgent ; 20 s = chat ; 30 s = API Purchase de secours ; local LoRA = pas ce timeout.
- `run_agent` = sandbox direct, pas MCP. C'est écrit dans `purchase_server.py` 1–4.
- `labels = input_ids` : `finetune_purchase_llm.py` 37. Loss sur toute la séquence.
- Température 0.2 + `do_sample=False` : warning Transformers, température ignorée.